

# Side-Channel Sensitivity Analysis on HQC: Towards a Fully Masked Implementation

Guillaume Goy<sup>1</sup>, Maxime Spyropoulos<sup>2,3</sup>, Nicolas Aragon<sup>1</sup>, Philippe Gaborit<sup>1</sup>, Renaud Pacalet<sup>2</sup>, Fabrice Perion<sup>3</sup>, Laurent Sauvage<sup>2</sup> and David Vigilant<sup>3</sup>

<sup>1</sup> XLIM, University of Limoges, Limoges, France

`{guillaume.goy,nicolas.aragon,gaborit}@unilim.fr`

<sup>2</sup> LTCI, Télécom Paris, Institut Polytechnique de Paris, France

`{maxime.spyropoulos,renaud.pacalet,laurent.sauvage}@telecom-paris.fr`

<sup>3</sup> Thales, France

`{fabrice.perion,david.vigilant}@thalgroup.fr`

**Abstract.** Hamming Quasi-Cyclic (HQC) has recently been officially selected for standardization by NIST as a post-quantum KEM alternative to ML-KEM. This milestone raises new requirements, in particular the need to design and deploy secure implementations of the scheme. This paper presents two major contributions to secure HQC against Side-Channel Attacks (SCAs). First, we present a detailed sensitivity analysis of HQC, highlighting the critical variables and critical internal functions that need to be protected. Second and main contribution, we propose the first fully masked HQC implementation at any order. It is also the first PQC masked implementation that is formally proved to be secure in the MIMO-SNI security model. This security, introduced by Cassiers and Standaert in 2020, ensures the security of gadgets composition against propagating probes. In this paper, we provide benchmarks of our implementation, showing that our masked implementation is competitive in the state-of-the-art masked PQC implementations.

**Keywords:** Hamming Quasi-Cyclic · Side-Channel Attack · Sensitivity analysis · Masked implementation

## 1 Introduction

The threat posed by quantum computers to cryptographic security has spurred research into Post-Quantum Cryptography (PQC). In 2016, the U.S. National Institute of Standards and Technology (NIST) launched a competition to standardize PQC schemes. By September 2022 [AAA<sup>+</sup>22], NIST had selected three lattice-based algorithms (Kyber [BDK<sup>+</sup>18], Dilithium [DKL<sup>+</sup>18], Falcon [PFH<sup>+</sup>20]) and one hash-based algorithm (SPHINCS+ [BHK<sup>+</sup>19]) as standards. At this time, NIST extended the contest to a fourth round to select new algorithms not based on lattices, aiming to increase the *diversity of computational hardness assumptions*.

Hamming Quasi-Cyclic (HQC) [AMAB<sup>+</sup>17] was one of the four alternate candidates and has recently been selected by NIST as a new PQC standard [AAC<sup>+</sup>23]. Although BIKE [ABB<sup>+</sup>17] offers smaller public key and ciphertext sizes, HQC was ultimately chosen due to its *stable Decryption Failure Rate (DFR) analysis*. Since 2019, HQC has employed concatenated Reed-Muller and Reed-Solomon codes as its public code structure, enabling a *precise analysis of the distribution of the error vector* [AGZ20] and, consequently, a precise DFR analysis. HQC is a Key Encapsulation Mechanism (KEM) built on a Public

Key Encryption (PKE) scheme, thanks to the HHK transform [HHK17], a variant of the Fujisaki-Okamoto transform [FO99]. Its security relies only on the hardness of solving the Quasi-Cyclic Syndrome Decoding (QCS) problem, without hiding the structure of an error-correcting code (unlike the Classic McEliece framework [BCL<sup>+</sup>]).

**Prior art regarding Side-Channel Attacks on HQC** Although HQC is secure against quantum adversaries, its implementations have been shown to be vulnerable to physical attacks, particularly Side-Channel Attacks (SCAs). The first implementations of HQC decoders were not constant-time and thus vulnerable to timing attacks [WTBB<sup>+</sup>20][PT19][HSC<sup>+</sup>23]. In addition, in 2022, Guo et al. [GJ20] showed that the rejection sampling process, used to generate small Hamming weight vectors, was also sensitive to timing attack. In consequence, the HQC specification has been corrected to become constant-time and prevent these attacks.

Since 2019, several side-channel attacks have been developed against HQC. The first category of attacks [SRSWZ20][SHR<sup>+</sup>22][GLG22a][PRJB][BMG<sup>+</sup>24] aims at recovering the secret key of HQC. These Chosen Ciphertext Attacks (CCA) train an oracle to obtain information about the secret key. They all strike the decoding step of HQC, [SHR<sup>+</sup>22] exploiting leakage from the Reed-Solomon outer code of HQC, while other attacks target the Reed-Muller inner code. With the development of new attack strategies, the number of physical traces (power, electro-magnetic) required has been significantly reduced. For instance since the first 50.000 required traces, Baisse et al. [BMG<sup>+</sup>24] showed that no more than 5 are sufficient with Belief Propagation (BP) tools. The second category of attacks [GLG22b][GMGL23][BMG<sup>+</sup>24] aims at recovering the shared key of HQC, which is the message of the PKE version of HQC. Since the shared key changes in each HQC session, these attacks must be performed with a single trace. They mostly target the first steps of the outer Reed-Solomon decoder of HQC, and are based on the BP framework, allowing to recover the shared key within minutes.

**Masking security models** The state of the art in SCAs against PQC, and more precisely against code-based cryptography, underscores the need for robust countermeasures, especially in embedded systems. A common countermeasure is masking, where each sensitive variable is split into  $d$  shares. With a masked implementation, an adversary with an access to  $t < d$  shares is unable to recover any information about the secret. Providing a masked implementation of a cryptographic scheme can be a significant challenge due to the complexity of ensuring security against SCAs. Nevertheless, Raccoon [dPKPR24] demonstrates that it is possible to design such masked implementations from the ground up with side-channel resistance as a primary goal. However, usually masked implementations are built using gadgets which are generic secured operations that satisfy certain composition and security-preserving properties. To study the formal security of gadgets composition, various security models have been introduced, among which the most widely adopted are the Non-Interference (NI) and Strong Non-Interference (SNI) models.

However, in 2020, Cassiers and Standaert [CS20] demonstrated that the composition of NI and SNI-secure gadgets does not necessarily guarantee secure gadget composition. They identified vulnerabilities resulting from what they called *propagating probes*, which can undermine the composition of such gadgets. To address this issue, they introduced two new security models called Multiple-Input-Multiple-Output Strong Non Interference (MIMO-SNI) and Probe Isolating Non Interference (PINI), which explicitly protect against propagating probe vulnerabilities. Within this framework, they also proposed a method to prove that an algorithm satisfies MIMO-SNI-security and showed that any MIMO-SNI-secure algorithm is also PINI-secure.

**Prior art regarding Countermeasures and secure implementations** HQC is not the only post-quantum cryptosystem requiring protection against side-channel leakage. Prior work has explored masking implementations for other schemes:

Migliore et al. [MGTF19] proposed a masked version of the lattice-based signature scheme Dilithium in 2019. Three years later, Azouaoui et al. [ABC<sup>+</sup>22a] identified two flaws in this implementation: (i) unmasked sensitive variables, creating a security gap, and (ii) unnecessary masking of public information, degrading performance. Azouaoui et al. improved the sensitivity analysis by categorizing intermediate computations and their security requirements.

Regarding Kyber, the other NIST-selected Key Exchange Mechanism (KEM) standard, several studies have referenced masked implementations or accelerations of masked operations, particularly focusing on the Number Theoretic Transform (NTT). Notably, a number of works have proposed first-order masked implementations, such as [HKL<sup>+</sup>22] and [FVBR<sup>+</sup>22], the latter of which also includes first-order masking for SABER and KECCAK. For higher-order masking, Bos et al. [BGR<sup>+</sup>21] introduced a generic masking scheme that is provably secure at any orders under the SNI model. In 2022, Bronchain and Cassiers [BC22] Kyber implementation achieves PINI security level.

In 2024, Demange et al. [DR24], introduced a NI secure masked implementation for BIKE, using only Boolean masking (avoiding arithmetic masking common in lattice-based schemes). The advantages of this choice are twofold: (i) it does not require mask conversions, which are known to be of high complexity, and (ii) BIKE is mainly based on binary operations, which makes boolean masking more natural and efficient.

Also in 2024, Spyropoulos et al. [SVP<sup>+</sup>25] presented a partially masked HQC implementation, covering key generation. Their approach uses a masked Barrett reduction with a NI security proof, showing that a hybrid arithmetic/Boolean masking is more efficient than pure Boolean masking.

This brings us the following questions: *Can we provide a strong sensitivity analysis for variables of HQC? Can we design a fully masked implementation of HQC secured in the MIMO-SNI security model?*

**Contributions** This paper presents two main contributions:

1. We perform a full sensitivity analysis of HQC operations. Based on the side-channel attacks background on HQC, we highlight that most variables and functions must be secured to prevent side-channel attacks. We show that the knowledge of variables leads either to a trivial secret recovery, or to a reduced cryptographic security of HQC.
2. Our primary contribution is the fully masked implementation of HQC. We protect each sensitive operation of HQC with secured functions such as a secured multiplication in Galois Field (`gf_mul`) or Fast Fourier Transform (`fft`). We show the security of our secured functions by providing evidence that they satisfy the MIMO-SNI conditions. To the best of our knowledge, this work presents the first masked implementation of a cryptographic scheme that is formally proved to be secure under the MIMO-SNI model, which strengthens the security guarantees compared to the widely used NI model. This proof is further supported by a practical side-channel analysis. We also provide a performance evaluation of our implementation and show that, despite the stronger security assumptions, it remains competitive with existing state-of-the-art masked implementations, including the masked implementation of BIKE [DR24].

**Paper organization** In Section 2, we present security models, notably the MIMO-SNI security model, and masking backgrounds. In Section 3, we recall HQC's main operations

and security assumptions. [Section 4](#) is dedicated to the sensitivity analysis of HQC variables and functions. Finally, [Section 5](#) presents the specificities of our fully masked implementation, along with a Test Vector Leakage Assessment (TVLA) and benchmark results. We have three appendices: [Appendix A](#) for additional benchmarks, [Appendix B](#) which introduces a new theoretical approach to exploit leakage from Reed-Solomon decoding step; and lastly, [Appendix C](#), which presents an ANOVA leakage analysis that complements the TVLA.

## 2 Masking background

Masking is a widely used countermeasure to protect implementations of sensitive algorithms against side-channel attacks. It consists in splitting any sensitive variable into  $d$  shares,  $d - 1$  of which are chosen uniformly at random, and the last one such that the “sharing relation” is satisfied. Any set of  $d - 1$  shares is thus always statistically independent of the secret variable. The most frequently encountered sharing relations are boolean  $x = \bigoplus_{i=1}^d x_i$

and arithmetic  $x = \sum_{i=1}^d x_i \bmod q$ , where  $x_i$  is the  $i$ -th share of  $x$ . We select the sharing relation depending on the nature of the operations to mask.

To maintain the security guarantees of masking throughout the execution of an algorithm, it is often necessary to perform refresh operations. A refresh consists in re-randomizing a masked value by generating new random shares that encode the same underlying secret. This operation ensures that the statistical distribution of the shares remains independent from one function to another, even if the same secret value is manipulated multiple times. Without refreshing, an attacker could exploit the correlation between reused shares across different masked operations to recover sensitive information. Refreshing is therefore a critical component in masked implementations, as it prevents such cumulative leakage and reinforces the security of the masked scheme against side-channel attacks.

### 2.1 Security in the $t$ -probing model

In the  $t$ -probing model an attacker has access to at most  $t$ -probes on the implementation, giving an access to  $t$  intermediate variables. By splitting the sensitive variables of an algorithm in  $d > t$  shares, and maintaining the statistical independence during the execution, attackers are prevented from learning information and the algorithm is secure in the  $t$ -probing model. Proving the theoretical security is a hard problem that does not scale well with the number of variables and shares. A common solution is to break down the algorithm into elementary operations referred to as “gadgets”, and to compose them to build larger functions. The composition must obey rules that preserve the security.

We recall two commonly used security models:

- A gadget is  $d$ -non-interfering ( $d$ -NI) if and only if any set of at most  $d$  probes can be perfectly simulated from at most  $d$  shares of each input.
- A gadget is  $d$ -strong non-interfering ( $d$ -SNI) [BBD<sup>+</sup>16] if and only if any set of at most  $d = d_{int} + d_{out}$  probes, where  $d_{int}$  is the number of internal probes and  $d_{out}$  is the number of output probes, can be perfectly simulated from at most  $d_{int}$  shares of each input. The  $d$ -SNI refresh gadget is crucial as it allows secure composition of gadgets [BBD<sup>+</sup>16]. We use it when needed to refresh the output of a  $d$ -NI gadget and turn it into a  $d$ -SNI gadget.

A well known result is that a gadget over  $d > t$  shares satisfying one of these two security models is  $t$ -probing secure.

## 2.2 MIMO-SNI security

MIMO-SNI is a notion proposed by Cassiers and Standaert [CS20] to allow the secure composition of Multiple Inputs (MI) and Multiple Outputs (MO) gadgets. It's an extension of SNI they developed after observing that the composition rules defined in [BBD<sup>+</sup>16] were not sufficient when considering MIMO gadgets. Cassiers and Standaert also introduce a generic method to prove that a gadget composition satisfies the MIMO-SNI property. Their approach relies on a Directed Acyclic Graph (DAG) capturing the interconnections between gadgets: the computation graph.

To ensure structural correctness, the computation graph forbids multiple edges being connected to the same gadget output or to a single input. In cases where an output needs to be reused, the authors introduce special nodes called **Split**. These nodes have one input and  $n$  identical outputs, and they perform no operation.

The composition involves three types of components: (i) NI gadgets with a single output, (ii) SNI refresh gadgets, and (iii) **Split** gadgets. Any SNI gadget is modeled as a composition of a NI gadget followed by a SNI refresh.

By leveraging the connection between the computation graph and the probe propagation framework, and exploiting the fact that SNI refresh gadgets block the propagation of probes, the authors propose a simplification step. This consists in removing all SNI refresh gadgets and their incident edges from the graph. The internal probes of these refresh gadgets are conservatively reassigned to their inputs, preserving security equivalence in the probing model. This results in a **simplified graph** that remains faithful to the original structure from a security standpoint.

Finally, the authors demonstrate that the gadget composition corresponding to this simplified graph satisfies the MIMO-SNI property if the following three conditions are satisfied:

1. There exists at most one path between any pair of nodes  $u$  and  $v$ .
2. For any pair of output nodes  $u_1$  and  $u_2$ , there is no node  $v$  such that there is a path from  $v$  to  $u_1$  and a path from  $v$  to  $u_2$ .
3. For any pair of input nodes  $u_1$  and  $u_2$ , there is no node  $v$  such that there exists a path from  $u_1$  to  $v$  and a path from  $u_2$  to  $v$ .

By proving that each individual gadget in the implementation satisfies the MIMO-SNI property, one can guarantee, by the gadget composition property, that the entire masked algorithm is in the MIMO-SNI security model. This composition property is a key advantage of the framework, as it enable scalable and modular design of secured masked implementations. Furthermore, the methodology to ensure that a gadget is MIMO-SNI is well understood, and this approach will be systematically applied to each of the functions of HQC requiring protection.

By proving the MIMO-SNI property for each gadget, we ensure that the masked version of HQC follows the MIMO-SNI security model. Not only is the final masked HQC implementation protected against side-channel attacks under the  $t$ -probing model, but it is also resilient to the propagating probes attacks.

## 3 Reminders on Hamming Quasi-Cyclic

Hamming Quasi-Cyclic (HQC) [AMAB<sup>+</sup>17] is a code-based post-quantum Key Encapsulation Mechanism (KEM). HQC was recently selected by the National Institute of Standard and Technology (NIST) to become a new standard for Post-Quantum Cryptography (PQC) [AAC<sup>+</sup>25]. HQC has been noticed especially for its “*DFR analysis*” which “*has remained stable throughout the NIST PQC standardization process*”.

HQC is a KEM based on a Public Key Encryption (PKE) scheme, thanks to the HHK transform [HHK17], a variant of the well-known Fujisaki-Okamoto transform [FO99]. HQC uses two codes: (i) a random code to ensure the security of secret keys and (ii) an efficient public code for encryption and decryption.

The PKE version of HQC (HQC-PKE) is described by three algorithms: Key Generation (algorithm 1), Encryption (algorithm 2) and Decryption (algorithm 3). These algorithms are summarized in Figure 1 where  $\mathcal{R}$  denotes the polynomial ring  $\mathbb{F}_2[X]/(X^n - 1)$ ,  $\text{HW}$  is the Hamming weight, and  $\mathcal{R}_\omega = \{\mathbf{x} \in \mathcal{R} ; \text{HW}(\mathbf{x}) = \omega\}$ . The notation  $\overset{\$}{\leftarrow} \theta \mathcal{R}_{\omega_e}$  means that the random sampling of an element from  $\mathcal{R}_{\omega_e}$  is seeded with  $\theta$ .

| Algorithm 1: KeyGen                                                                                                                                                                                                                                                                                                                                                                               | Algorithm 2: Encrypt                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Algorithm 3: Decrypt                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> HQC parameters<br><b>Output:</b> (sk, pk)<br>1 $\mathbf{h} \overset{\$}{\leftarrow} \mathcal{R}$<br>2 $(\mathbf{x}, \mathbf{y}) \overset{\$}{\leftarrow} \mathcal{R}_\omega^2$<br>3 $\text{sk} \leftarrow (\mathbf{x}, \mathbf{y})$<br>4 $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{h}\mathbf{y}$<br>5 $\text{pk} \leftarrow (\mathbf{h}, \mathbf{s})$<br>6 <b>return</b> (sk, pk) | <b>Input:</b> pk, m, $\theta$<br><b>Output:</b> ct<br>1 $\mathbf{e} \overset{\$}{\leftarrow} \theta \mathcal{R}_{\omega_e}$<br>2 $(\mathbf{r}_1, \mathbf{r}_2) \overset{\$}{\leftarrow} \theta \mathcal{R}_{\omega_r}^2$<br>3 $\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$<br>4 $\mathbf{v} \leftarrow \mathcal{C}.\text{Encode}(\mathbf{m}) + \mathbf{s}\mathbf{r}_2 + \mathbf{e}$<br>5 $\text{ct} \leftarrow (\mathbf{u}, \mathbf{v})$<br>6 <b>return</b> ct | <b>Input:</b> sk, ct<br><b>Output:</b> Message m<br>1 <b>return</b> $\mathcal{C}.\text{Decode}(\mathbf{v} - \mathbf{u}\mathbf{y})$ |

Table 1: HQC parameters from [AMAB<sup>+</sup>17].

| Security | mul | $k_1$ | $k_2$ | $n_1$ | $n_2$ | $n_1 n_2$ | $n$   | $l$ | $\omega$ | $\omega_e = \omega_r$ |
|----------|-----|-------|-------|-------|-------|-----------|-------|-----|----------|-----------------------|
| HQC-128  | 3   | 16    | 8     | 46    | 384   | 17664     | 17669 | 5   | 66       | 75                    |
| HQC-192  | 5   | 24    | 8     | 56    | 640   | 35840     | 35851 | 11  | 100      | 114                   |
| HQC-256  | 5   | 32    | 8     | 90    | 640   | 57600     | 57637 | 37  | 131      | 149                   |

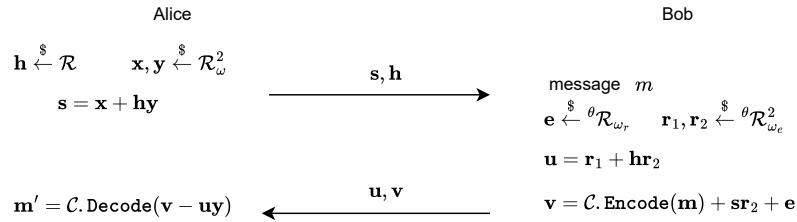


Figure 1: Summarized HQC-PKE scheme

In HQC-KEM, the encryption function is de-randomized thanks to a hash of the message and public key used as a seed. This allows to re-encrypt the message after decryption to verify if the received ciphertext is equal to the computed one (this step is required by the HHK transform). The session key, that corresponds to a hash of the concatenation of the message and the ciphertext, is shared only if the equality is verified. The KEM is composed of Encapsulation (algorithm 4) and Decapsulation (algorithm 5), where  $\mathcal{G}$ ,  $\mathcal{H}$  and  $\mathcal{K}$  are cryptographically secure hash functions and  $\text{pk}_{32}$  represents the 32 first bytes of pk.



| Algorithm 4: Encapsulate                                                                                                                                                                                                                                                                                                                                                                                         | Algorithm 5: Decapsulate                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> $\text{pk}$<br>1 $\mathbf{m} \xleftarrow{\$} \mathbb{F}_2^k$<br>2 $\theta \leftarrow \mathcal{G}(\mathbf{m} \parallel \text{pk}_{32} \parallel \text{salt})$<br>3 $\mathbf{c} \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}, \theta)$<br>4 $K \leftarrow \mathcal{K}(\mathbf{m}, \mathbf{c})$<br>5 $\mathbf{d} \leftarrow \mathcal{H}(\mathbf{m})$<br>6 <b>return</b> $(\mathbf{c}, \mathbf{d})$ | <b>Input:</b> $\text{sk}, \mathbf{c}, \mathbf{d}$<br>1 $\mathbf{m}' \leftarrow \text{Decrypt}(\text{sk}, \mathbf{c})$<br>2 $\theta' \leftarrow \mathcal{G}(\mathbf{m}')$<br>3 $\mathbf{c}' \leftarrow \text{Encrypt}(\text{pk}, \mathbf{m}', \theta')$<br>4 <b>if</b> $\mathbf{c} \neq \mathbf{c}'$ <b>or</b> $\mathbf{d} \neq \mathcal{H}(\mathbf{m}')$ <b>then</b><br>5 <b>return</b> $\perp$<br>6 <b>else</b><br>7 <b>return</b> $\mathcal{K}(\mathbf{m}, \mathbf{c})$ |

### 3.1 HQC security

HQC-PKE has been proved IND-CPA secure, and the application of the HHK transform, combined with the strong DFR analysis, allows HQC-KEM to reach IND-CCA2 security. The security of HQC relies on the hardness of solving the Quasi-Cyclic Syndrome Decoding (*QCSD*) problem [BMVT78] without any other assumptions. Security reduction proofs can be found in section 5 of HQC documentation [AMAB<sup>+</sup>17]. First, the secret keys security relies on solving an instance of the  $2 - \text{QCSD} - \mathcal{P}$  problem:

$$(I_n \quad \text{rot}(\mathbf{h})) \cdot (\mathbf{x}, \mathbf{y})^T = \mathbf{s} \quad (1)$$

Second, the IND-CPA security of HQC relies on the ability of solving two instances of the Quasi-cyclic Syndrome decoding:

$$\begin{pmatrix} I_n & 0 & \text{rot}(\mathbf{h}) \\ 0 & I_n & \text{rot}(\mathbf{s}) \end{pmatrix} \cdot (\mathbf{r}_1, \mathbf{e}, \mathbf{r}_2)^T = (\mathbf{u}, \mathbf{v} - \mathbf{m}\mathbf{G}) \quad (2)$$

Solving this problem is equivalent to solving an instance of the  $3 - \text{QCSD} - \mathcal{PT}$  problem.

## 4 HQC Side-Channel Sensitivity Analysis

In this section, we provide a strong side-channel sensitivity analysis of variables of HQC. We first represent variables and functions of HQC-KEM in an oriented graph (see Figure 2). A variable node is connected to a function node with a descending arrow if it is an input or with an ascending arrow if it is an output. The purpose of this section is to decide if a variable leaks information about a secret and must thus be masked. Variables holding a secret key ( $\text{sk}$ ) or part of the shared key are considered as sensitive while variables holding part of the public key ( $\text{pk}$ ) or ciphertext ( $\text{ct}$ ) are considered as non-sensitive. For any other variable  $v$ , we show that the knowledge of  $v$  (denoted  $\mathcal{K}(v)$ ) leads to the recovery of another sensitive data or secret. We summarize our analyzes in Figure 2 where sensitive variables and functions are red and non-sensitive variables and functions are green.

### 4.1 Variables sensitivity

The secret key of HQC is the pair of small Hamming weight vectors  $(\mathbf{x}, \mathbf{y})$ . The shared key  $K$  is derived from the message  $\mathbf{m}$ , denoted  $\mathbf{m}'$  in the Decapsulation process. The public information in HQC is the public key  $(\mathbf{h}, \mathbf{s})$ , the public known code represented by generator matrix  $\mathbf{G}$  and parity check matrix  $\mathbf{H}$ , and ciphertext vectors  $\mathbf{c} = (\mathbf{u}, \mathbf{v})$  and  $\mathbf{d}$ .

**KeyGen** The following variables must be secured: (i)  $(\mathbf{x}, \mathbf{y})$  since it is the secret key and (ii)  $\mathbf{t}_0$  since  $\mathcal{K}(\mathbf{t}_0) \implies \mathcal{K}(\mathbf{x})$  by:  $\mathbf{x} := \mathbf{s} - \mathbf{t}_0$ .

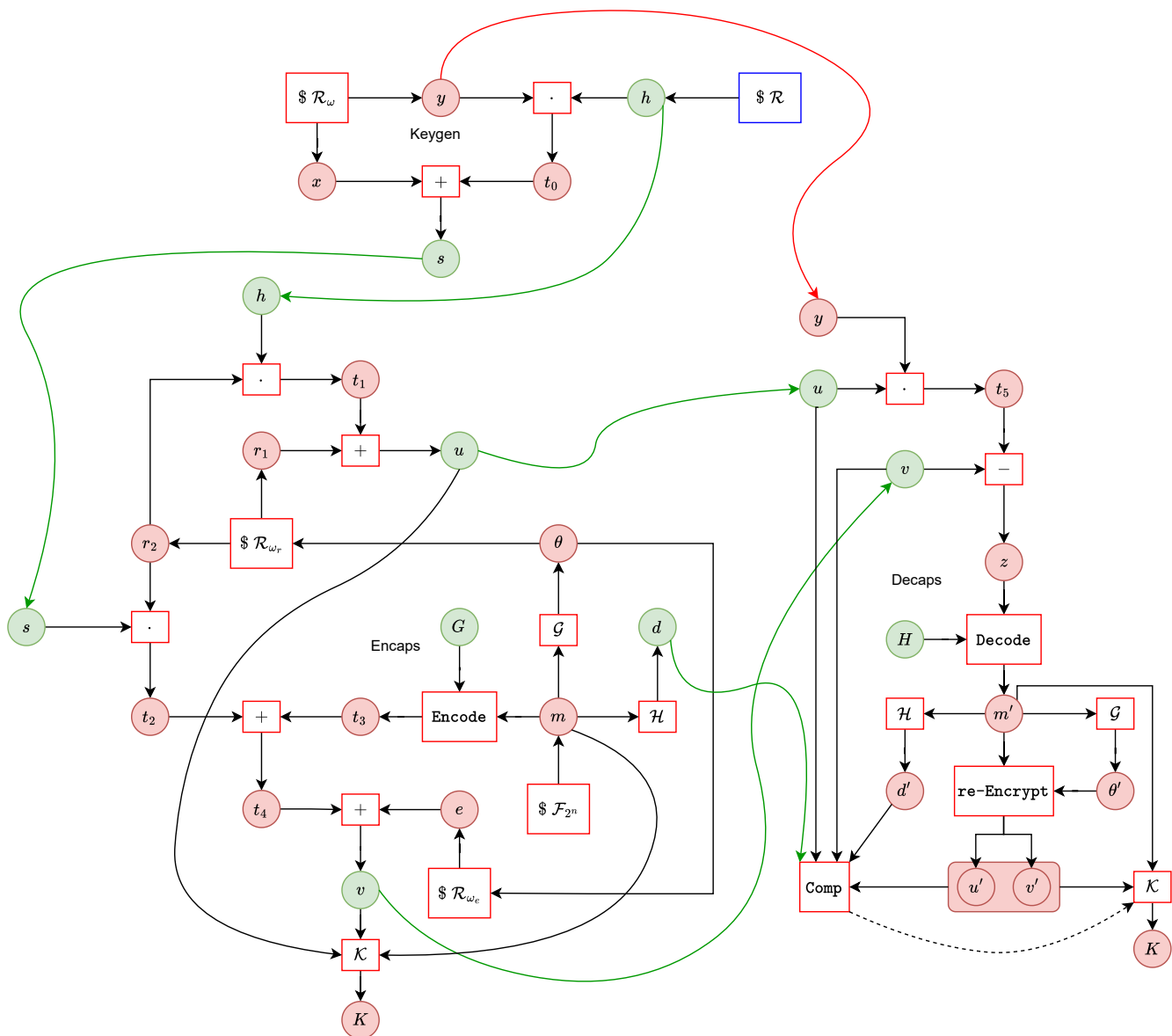


Figure 2: Variables and functions of HQC KEM, **red**: sensitive, **blue**: non-sensitive, **green**: public data



**Encaps** The Encaps function involves the generation of ephemeral random values  $\mathbf{r}_1, \mathbf{r}_2$  and  $\mathbf{e}$ . From Section 3, we know that the security reduction relies on the hardness of solving an instance of Equation 2. The knowledge of any of the three vectors  $\mathbf{r}_1, \mathbf{r}_2$  or  $\mathbf{e}$  dramatically decreases the complexity of this operation which can be solved by inverting the residual matrix. We thus have that:

$$\mathcal{K}(\mathbf{r}_1) \iff \mathcal{K}(\mathbf{r}_2) \iff \mathcal{K}(\mathbf{e}) \quad (3)$$

any of them would break the IND-CPA security of HQC, they must be secured. The following other variables must also be secured: (i)  $K$  since it is the shared key. (ii)  $\mathbf{m}$  since  $\mathcal{K}$  is a publicly known hash function, so  $\mathcal{K}(\mathbf{m}) \implies \mathcal{K}(K)$ . (iii)  $\theta$  since it allows to generate all encryption random values  $\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2$ . (iv)  $\mathbf{t}_1$  since  $\mathcal{K}(\mathbf{t}_1) \implies \mathcal{K}(\mathbf{r}_1)$  by:  $\mathbf{r}_1 := \mathbf{u} - \mathbf{t}_1$ . (v)  $\mathbf{t}_2$  since  $\mathcal{K}(\mathbf{t}_2) \implies \mathcal{K}(\mathbf{r}_2)$  by:  $\mathbf{r}_2 = \mathbf{s}^{-1}\mathbf{t}_2$ . (vi)  $\mathbf{t}_3$  since  $\mathcal{K}(\mathbf{t}_3) \implies \mathcal{K}(\mathbf{m})$  as  $\mathbf{t}_3 = \mathbf{m}\mathbf{G}$  can be reversed. (vii)  $\mathbf{t}_4$  since  $\mathcal{K}(\mathbf{t}_4) \implies \mathcal{K}(\mathbf{e})$ , by:  $\mathbf{e} = \mathbf{v} - \mathbf{t}_4$ .

**Decaps** The following variables must be secured: (i)  $K, \mathbf{y}$  as they are respectively the shared key and part of the secret key. (ii)  $\mathbf{m}'$  since  $\mathcal{K}$  is a publicly known hash function, so  $\mathcal{K}(\mathbf{m}') \implies \mathcal{K}(K)$ . (iii)  $\mathbf{z}$  since the decoder is publicly known and decoding  $\mathbf{z}$  allows  $\mathcal{K}(\mathbf{m}')$ . (iv)  $\mathbf{t}_5$  since  $\mathcal{K}(\mathbf{t}_5) \implies \mathcal{K}(\mathbf{z})$ , by:  $\mathbf{z} = \mathbf{v} - \mathbf{t}_5$ . (v)  $\theta'$  since it allows to generate all re-encryption random values  $\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2$ . (vi)  $\mathbf{u}'$  and  $\mathbf{v}'$  since the result of Comp could be used to build a CCA and bypass the re-encryption security.

**Re-encryption** We have shown that all variables manipulated during the encryption step are sensitive. In a typical use case of HQC, the re-encryption step processes the exact same variables as those used during encryption. It follows that the variables and functions involved in re-encryption must also be protected using the same security measures as those applied during encryption.

## 4.2 Functions sensitivity

Any function that manipulates at least one sensitive variable is considered a sensitive function. Each sensitive function must be transformed into a secure gadget to ensure that the manipulation of sensitive variables is carried out in a protected manner. These functions are: (i)  $+$  addition in characteristic 2. (ii)  $\cdot$  binary vector multiplication. (iii)  $\times$  binary matrix vector multiplication. (iv)  $\$R_\omega$  random sample from a given set (with Hamming weight constraint). (v)  $\mathcal{G}, \mathcal{K}$  and  $\mathcal{H}$  hash functions. (vi) Comp equality verification (inputs are secret and output must be secured to prevent CCA). (vii) HQC Decoder is a complex operation, the next section is dedicated to its sensitivity analysis.

## 4.3 Sensitivity of the HQC decoder

The decoder of HQC is a concatenation of Reed-Muller and Reed-Solomon decoders. It is a complex function composed of numerous operations.

Many side-channel attacks [GLG22a][PRJB][BMG<sup>+</sup>24] exploit some leakage of the Reed-Muller decoder at various steps of the processing (from ExpandAndSum to the FindPeaks operation). It follows that all operations of this decoder must be protected to thwart these attacks.

On the other hand, [GLG22b] and [GMGL23] showed the Reed-Solomon syndrome computation leaks information that lead to recover the shared key of HQC. The authors also showed that the encoding process of HQC can leak information about the shared key and thus also needs to be secured. Furthermore, the attack in [SHR<sup>+</sup>22] demonstrates that the whole Reed-Solomon decoder can be exploited for key recovery. In Appendix B,

we introduce a new theoretical approach, that confirms the need to protect any operation of the Reed-Solomon code, not only the syndrome computation. Hence, each operation of the HQC encoder and decoder needs to be masked to ensure the global security of the scheme. Figure 3 and Figure 4 presents the HQC encoder and decoder functions.

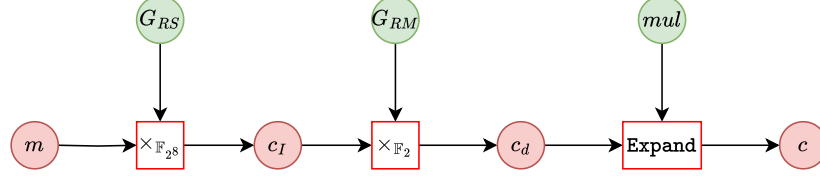


Figure 3: Variables and functions of the HQC encoder, red: sensitive, green: public data

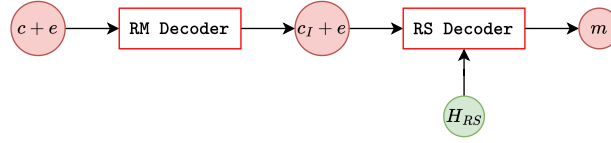


Figure 4: Variables and functions of the HQC decoder, red: sensitive, green: public data

**Reed-Muller decoder** The Reed-Muller code used in HQC is a first order code that can be seen as a Hadamard code, and HQC uses a Hadamard decoder for the Reed-Muller decoding. The decoder operates in four steps:

1. The first step, called **ExpandAndSum**, removes the multiplicity (part of a repetition code) in its input binary vector, and outputs a vector of natural integers.
2. The second step, the fast Hadamard transform (FHT), is the main operation and consists of natural additions and subtractions.
3. The third step corrects the FHT by slightly modifying the first component of the output vector.
4. The fourth and last step, **FindPeaks**, finds the sign and location of the component of the output vector with maximum absolute value, from which the final output value is computed.

FHT is a linear operation, as well as **ExpandAndSum** and the FHT correction. However the last step, **FindPeaks**, is nonlinear.

**Reed-Solomon decoder** The Reed-Solomon decoder of HQC decodes by computing the roots of the error location polynomial as described in [MS77]. The first step is the syndrome computation. It has already been targeted by side-channel attacks [GLG22b][GMGL23] and must be protected. The last step mostly processes zeros due to the low decryption failure rate after the Reed-Muller decoder. Indeed, Schamberger et al. [SHR<sup>+</sup>22] demonstrated that leakage from the Reed-Solomon decoder can reveal information about the HQC secret key. Their attack leverages the fact that a decoding failure in the Reed-Solomon code correlates directly with a failure in the Reed-Muller decoder. As a result, monitoring the physical behavior of the Reed-Solomon decoder can leak critical information about the secret key. Their full attack is completed by an Information Set Decoding (ISD) phase, which ultimately enables recovery of the entire secret key. We also present a new approach (see Appendix B) to mount a key recovery side-channel attack exploiting the Reed-Solomon leakages.

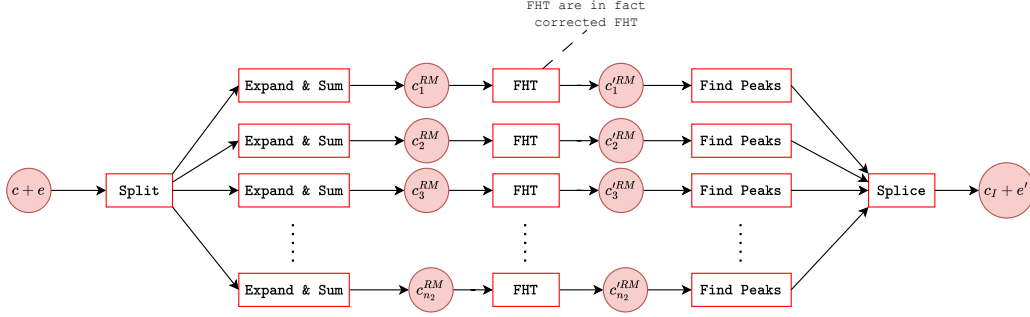


Figure 5: Variables and functions of HQC Reed-Muller decoder, red: sensitive

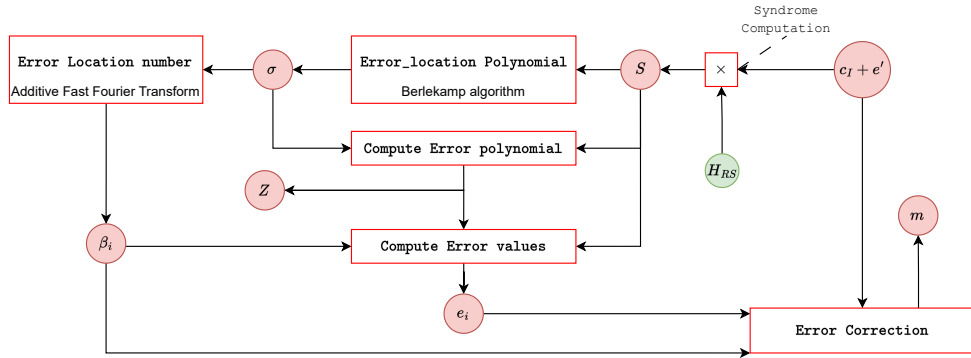


Figure 6: Variables and functions of HQC Reed-Solomon decoder, red: sensitive, green: public data

**Wrap-up** The attack presented in [SHR<sup>+</sup>22], together with our analysis in Appendix B, highlights the necessity of protecting the entire Reed-Solomon decoder against side-channel attacks. Moreover, numerous state-of-the-art attacks [GLG22a][PRJB][BMG<sup>+</sup>24] targeting the Reed-Muller decoding step demonstrate that this component must also be secured. In addition, we have established that all variables and functions involved in HQC (excluding public key generation and manipulation) are sensitive (see Figure 2), in the sense that knowledge of any one of them could lead to the disclosure of a cryptographic secret. It follows that all such elements must be properly protected to ensure the overall security of the implementation.

## 5 Implementation details

### 5.1 MIMO-SNI security proof

To complete our fully masked HQC implementation, we use some generic gadgets (see Table 2) from the state-of-the-art. Some generic gadgets are used *as-in* for our implementation, while we re-implemented some of them.

Table 2: Secured gadgets used in our implementation

| Gadget               |                              | Security        | Function                         |
|----------------------|------------------------------|-----------------|----------------------------------|
| Sec <sub>&amp;</sub> | [CGTV15]                     | SNI             | and operator                     |
| SecAdd               | [CGTV15]                     | NI              | Addition in $\mathbb{Z}$         |
| Refresh              | [BCPZ16]                     | SNI             | Refresh mask                     |
| ConvertAtoB          | [CGV14]                      | NI <sup>1</sup> | Convert Arithmetic to Boolean    |
| ConvertBtoA          | [CGV14]                      | NI <sup>1</sup> | Convert Boolean to Arithmetic    |
| Sec <sub>=</sub>     | [DR24]                       | NI              | Verify masked variables equality |
| Sec <sub>if</sub>    | [DR24]                       | NI              | Conditional branch               |
| Sec <sub>SHA-3</sub> | [CGL <sup>+</sup> 24][Ger24] | NI              | Secured SHA-3                    |

To ensure that the functions we develop satisfy the security requirements of the MIMO-SNI model, we must verify specific structural properties on the simplified graph of the algorithm, as outlined in Section 2. To facilitate this process, we developed a tool that automatically generates a graphical representation of an algorithm where the nodes are the elementary operations and the edges represent the data flows. This ad hoc solution is built upon a Python parser inspired by whitebox analysis methods. The parser scans a pre-compiled C file in order to extract function calls and link their inputs and outputs together. It then creates a file containing the graph's vertexes and edges in dot format, that we plot with *igraph*. Using this tool, we are able to model and analyze any algorithm and transform it into a MIMO-SNI gadget. We used this methodology to secure all algorithms of HQC (see Table 3). As an example, Figure 7 shows the output of our representation tool for the Barrett reduction used in HQC.

Verifying that the three security conditions are met then reduces to a refresh placement problem within the graph of the algorithm.

In masked implementations, refresh operations are computationally expensive. Our tool was specifically developed to help us with their placement. It can verify that the three conditions for compliance with the MIMO-SNI model are met. If not, the graphical representation allows us to identify the most efficient placement of refresh operations, with the dual objective of minimizing their number and ensuring that the required security properties are satisfied. As a result, we provide evidence that our masked implementation of HQC satisfies the conditions required by the MIMO-SNI security model.

<sup>1</sup>We re-implemented these functions from scratch following the NI composition rules.



Table 3: Secured HQC functions

| Algorithm                     | Security | Function                       |
|-------------------------------|----------|--------------------------------|
| $\text{Sec}_{\text{gf\_mul}}$ | MIMO-SNI | Multiplication in Galois Field |
| $\text{Sec}_{\text{fft}}$     | MIMO-SNI | Fast Fourier Transform         |
| $\text{Sec}_{\text{barrett}}$ | MIMO-SNI | Barrett reduction              |
| $\text{Sec}_{\text{enc\_RS}}$ | MIMO-SNI | Reed-Solomon Encoder           |
| $\text{Sec}_{\text{dec\_RS}}$ | MIMO-SNI | Reed-Solomon Decoder           |
| $\text{Sec}_{\text{enc\_RM}}$ | MIMO-SNI | Reed-Muller Encoder            |
| $\text{Sec}_{\text{dec\_RM}}$ | MIMO-SNI | Reed-Muller Decoder            |

## 5.2 Test Vector Leakage Assessment (TVLA)

**Welch  $t$ -test** is a statistical method used to assess whether there is a significant difference between the means of two independent samples. It constitutes a more robust adaptation of Student’s  $t$ -test, particularly well-suited to scenarios in which the variances of the two populations under comparison are unequal.

If  $\mathcal{S}_0$  and  $\mathcal{S}_1$  denote two sets of respective cardinalities  $n_0$  and  $n_1$ , with means  $\mu_0$  and  $\mu_1$ , and variances  $\sigma_0^2$  and  $\sigma_1^2$ . The Welch’s  $t$ -test result, commonly known as  $t$ -value is computed as follows:

$$t = \frac{\mu_0 - \mu_1}{\sqrt{\left(\frac{\sigma_0^2}{n_0} + \frac{\sigma_1^2}{n_1}\right)}} \quad (4)$$

It is statistically proven that a threshold value of  $|t| = 4.5$  represents a classifier with a confidence greater than 99.999%. An absolute  $t$ -value exceeding this threshold indicates that the two distributions  $\mathcal{S}_0$  and  $\mathcal{S}_1$  are statistically distinguishable [DCE16]. Otherwise, an absolute  $t$ -value below this threshold shows that the two sets cannot be distinguished by a linear classifier.

We tested the practical security of our implementation by targeting the masked Reed-Solomon decoder. It was chosen because of its sensibility to SCA, as recalled in our sensitivity analysis (see Section 4). To conduct our experiments we used the STM32 Nucleo-F439ZI development board which is equipped with an ARM Cortex-M4 core running at 168 MHz. We performed a fixed-vs-random  $t$ -test by recording two sets of electro-magnetic traces: one with a fixed input and the other with random inputs. For this first test we set all masks to zero, to virtually unmask our implementation in order to assess its leakage level in an unprotected setting. Then we ran a second fixed-vs-random  $t$ -test, this time with random masks to find out if there was any leakage in the typical use case of our solution. The results are visible in Figure 8. The noticeable peak at  $1e6$  in Figure 8a corresponds to the moment when we create the codeword that will be decoded. This is not part of the decoder, but we wanted to capture what comes before, to assess the leakage level of the algorithm when it is not manipulating the codeword. This is an important information, because we can directly compare it to the apparent leakage level when using random masks. Both have a similar amplitude, contained between  $-4.5/+4.5$  (red horizontal lines), which comforts us in the relevance of our solution. In Figure 8b, small peaks are still visible around  $1e6$ . Again, they are due to the creation of the codeword, they are not part of the decoder, thus not significant in the leakage analysis. For completeness, we also ran an ANOVA leakage analysis (see Appendix C), which corroborates the conclusions we drew from the  $t$ -tests.

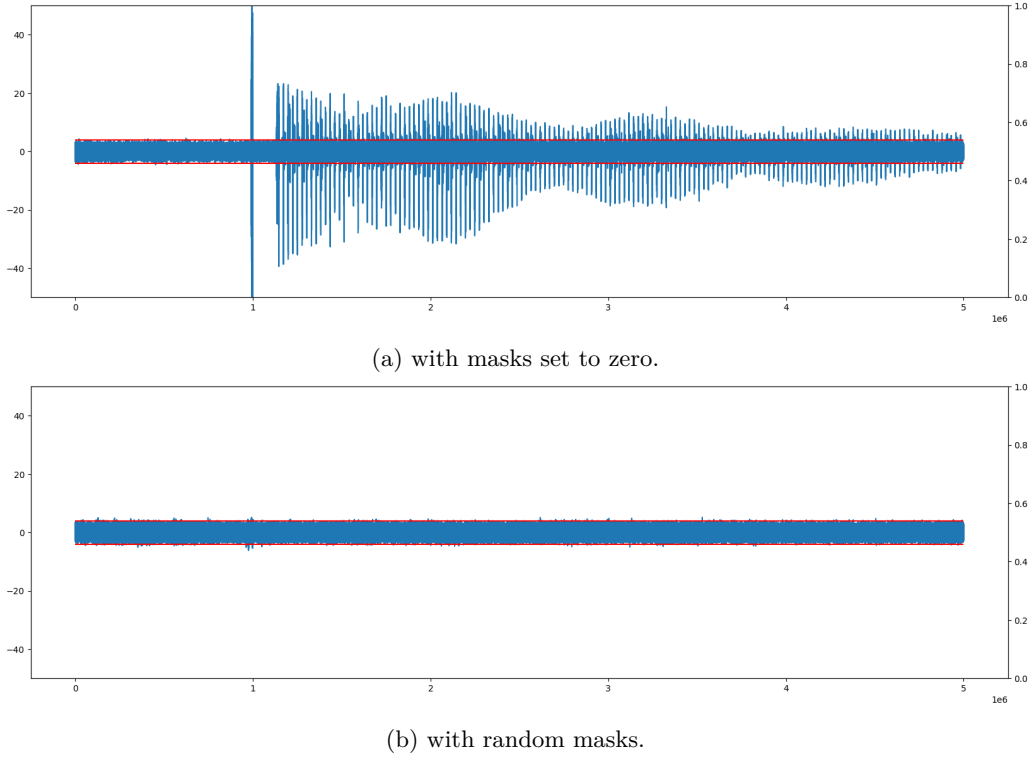


Figure 8: TVLA of the masked Barrett reduction with mask conversions.

### 5.3 Benchmark of HQC masked implementation

We performed a series of benchmarks on our masked implementations to compare the execution times for different numbers of shares and different security models. We conducted these experiments on a laptop equipped with an Intel Core Ultra 7 165U processor (14 logical CPUs), 16 GB of RAM, a 512 GB SSD, and running a Windows operating system. The processor was configured to maintain a fixed frequency of 2.69GHz, ensuring stable experimental conditions. To measure executions times, we directly accessed the processor’s time-stamp counter using the `rdtsc` instruction, which directly provides the number of CPU clock cycles. We compiled our HQC masked implementations using `gcc 11.4.0` with the `-O3` optimization flag. We also performed a series of additional benchmark with `-O0` optimization flag, which can be found in [Appendix A](#).

For each masking order from 0 to 7 (1 to 8 shares) we repeated the experiment 120 times, preceded by a warm-up step, to obtain an average time for KeyGen, Encaps, and Decaps. We did a benchmark for two different HQC implementations with two security models, NI and MIMO-SNI, to estimate the impact of increased security on performance. The results of the benchmarks are shown in [Table 4](#) for the MIMO-SNI security model and in [Table 5](#) for the NI security model. We also provide scaled values, with reference 1 for masking order 0, for easier comparison of the impact of masking orders.

The order-0 masking configuration corresponds to an implementation using a single share per variable. While this configuration provides no protection for sensitive variables, it serves as a baseline for comparing execution time scaling across implementations with masking structures.

As expected, increasing the security requirements introduces additional performance and computational overhead. We observe an overhead of approximately 5 to 10% for the MIMO-SNI implementation compared to the NI version. In this performance-security trade-



Table 4: Benchmarks of MIMO-SNI HQC masked implementation (in millions of cycles) with -O3 optimization flag.

| Order         | 0            | 1            | 2             | 3             | 4             | 5             | 6              | 7              |
|---------------|--------------|--------------|---------------|---------------|---------------|---------------|----------------|----------------|
| HQC-128       |              |              |               |               |               |               |                |                |
| KeyGen        | 6.3          | 43.7         | 95.1          | 247.0         | 328.1         | 447.1         | 644.4          | 845.3          |
| KeyGen scaled | $\times 1.0$ | $\times 7.0$ | $\times 15.1$ | $\times 39.3$ | $\times 52.2$ | $\times 71.2$ | $\times 102.6$ | $\times 134.6$ |
| Encaps        | 11.7         | 81.7         | 181.5         | 463.9         | 622.5         | 843.4         | 1230.8         | 1608.9         |
| Encaps scaled | $\times 1.0$ | $\times 7.0$ | $\times 15.6$ | $\times 39.8$ | $\times 53.4$ | $\times 72.4$ | $\times 105.6$ | $\times 138.1$ |
| Decaps        | 25.8         | 128.3        | 270.9         | 675.1         | 911.5         | 1220.5        | 1827.0         | 2374.5         |
| Decaps scaled | $\times 1.0$ | $\times 5.0$ | $\times 10.5$ | $\times 26.2$ | $\times 35.4$ | $\times 47.4$ | $\times 70.9$  | $\times 92.2$  |
| HQC-192       |              |              |               |               |               |               |                |                |
| KeyGen        | 19.0         | 122.5        | 262.9         | 690.8         | 916.1         | 1211.5        | 1787.7         | 2338.2         |
| KeyGen scaled | $\times 1.0$ | $\times 6.5$ | $\times 13.9$ | $\times 36.4$ | $\times 48.3$ | $\times 63.8$ | $\times 94.2$  | $\times 123.2$ |
| Encaps        | 35.2         | 227.5        | 492.9         | 1285.5        | 1718.8        | 2297.1        | 3378.0         | 4433.1         |
| Encaps scaled | $\times 1.0$ | $\times 6.5$ | $\times 14.0$ | $\times 36.5$ | $\times 48.9$ | $\times 65.3$ | $\times 96.0$  | $\times 126.0$ |
| Decaps        | 61.9         | 332.2        | 705.4         | 1799.7        | 2429.1        | 3231.9        | 4800.1         | 6267.4         |
| Decaps scaled | $\times 1.0$ | $\times 5.4$ | $\times 11.4$ | $\times 29.1$ | $\times 39.3$ | $\times 52.2$ | $\times 77.6$  | $\times 101.3$ |
| HQC-256       |              |              |               |               |               |               |                |                |
| KeyGen        | 36.9         | 232.3        | 509.1         | 1352.8        | 1779.7        | 2340.4        | 3484.1         | 4574.9         |
| KeyGen scaled | $\times 1.0$ | $\times 6.3$ | $\times 13.8$ | $\times 36.7$ | $\times 48.3$ | $\times 63.5$ | $\times 94.5$  | $\times 124.1$ |
| Encaps        | 67.7         | 428.2        | 947.3         | 2486.2        | 3303.9        | 4371.5        | 6509.0         | 8517.3         |
| Encaps scaled | $\times 1.0$ | $\times 6.3$ | $\times 14.0$ | $\times 36.7$ | $\times 48.8$ | $\times 64.5$ | $\times 96.1$  | $\times 125.7$ |
| Decaps        | 115.8        | 618.3        | 1340.8        | 3461.1        | 4630.2        | 6129.8        | 9162.8         | 11975.6        |
| Decaps scaled | $\times 1.0$ | $\times 5.3$ | $\times 11.6$ | $\times 29.9$ | $\times 40.0$ | $\times 52.9$ | $\times 79.1$  | $\times 103.4$ |

Table 5: Benchmarks of NI HQC masked implementation (in millions of cycles) with -O3 optimization flag.

| Order         | 0            | 1            | 2             | 3             | 4             | 5             | 6              | 7              |
|---------------|--------------|--------------|---------------|---------------|---------------|---------------|----------------|----------------|
| HQC-128       |              |              |               |               |               |               |                |                |
| KeyGen        | 6.2          | 43.0         | 93.6          | 239.6         | 318.4         | 421.4         | 620.8          | 817.6          |
| KeyGen scaled | $\times 1.0$ | $\times 7.0$ | $\times 15.2$ | $\times 38.8$ | $\times 51.6$ | $\times 68.2$ | $\times 100.5$ | $\times 132.4$ |
| Encaps        | 11.4         | 79.7         | 178.0         | 446.5         | 610.1         | 805.6         | 1179.8         | 1538.8         |
| Encaps scaled | $\times 1.0$ | $\times 7.0$ | $\times 15.6$ | $\times 39.1$ | $\times 53.5$ | $\times 70.6$ | $\times 103.4$ | $\times 134.8$ |
| Decaps        | 25.3         | 125.5        | 263.7         | 650.4         | 878.4         | 1170.7        | 1749.3         | 2264.9         |
| Decaps scaled | $\times 1.0$ | $\times 5.0$ | $\times 10.4$ | $\times 25.7$ | $\times 34.7$ | $\times 46.2$ | $\times 69.1$  | $\times 89.4$  |
| HQC-192       |              |              |               |               |               |               |                |                |
| KeyGen        | 18.3         | 117.0        | 257.2         | 670.2         | 878.8         | 1175.5        | 1735.3         | 2256.4         |
| KeyGen scaled | $\times 1.0$ | $\times 6.4$ | $\times 14.0$ | $\times 36.6$ | $\times 48.0$ | $\times 64.1$ | $\times 94.7$  | $\times 123.1$ |
| Encaps        | 34.0         | 217.0        | 483.1         | 1231.6        | 1648.2        | 2223.5        | 3277.5         | 4253.9         |
| Encaps scaled | $\times 1.0$ | $\times 6.4$ | $\times 14.2$ | $\times 36.2$ | $\times 48.4$ | $\times 65.3$ | $\times 96.3$  | $\times 125.0$ |
| Decaps        | 60.1         | 317.2        | 687.4         | 1726.1        | 2312.7        | 3117.0        | 4616.8         | 5996.5         |
| Decaps scaled | $\times 1.0$ | $\times 5.3$ | $\times 11.4$ | $\times 28.7$ | $\times 38.5$ | $\times 51.9$ | $\times 76.8$  | $\times 99.8$  |
| HQC-256       |              |              |               |               |               |               |                |                |
| KeyGen        | 35.8         | 227.5        | 501.8         | 1305.9        | 1718.0        | 2279.9        | 3364.8         | 4357.2         |
| KeyGen scaled | $\times 1.0$ | $\times 6.4$ | $\times 14.0$ | $\times 36.5$ | $\times 48.0$ | $\times 63.7$ | $\times 94.1$  | $\times 121.8$ |
| Encaps        | 65.3         | 417.8        | 928.0         | 2387.0        | 3171.0        | 4265.2        | 6345.3         | 8161.0         |
| Encaps scaled | $\times 1.0$ | $\times 6.4$ | $\times 14.2$ | $\times 36.6$ | $\times 48.6$ | $\times 65.4$ | $\times 97.2$  | $\times 125.1$ |
| Decaps        | 112.8        | 604.7        | 1310.9        | 3318.5        | 4453.5        | 5972.6        | 8783.2         | 11445.3        |
| Decaps scaled | $\times 1.0$ | $\times 5.4$ | $\times 11.6$ | $\times 29.4$ | $\times 39.5$ | $\times 52.9$ | $\times 77.8$  | $\times 101.4$ |

off, one may choose between (i) an implementation secure in the NI model, which remains comparable to most state-of-the-art masked PQC schemes [DR24, ABC<sup>+</sup>22b], or (ii) a more robust implementation that satisfies the MIMO-SNI security model. Nevertheless, even our MIMO-SNI-masked implementation remains competitive when compared to BIKE for the same masking order. For example, at masking order 5, HQC key generation requires 447 million cycles, compared to 853 million for BIKE [DR24]. For the same masking order, the encapsulation and decapsulation phases require 2063 million cycles for both NI BIKE and MIMO-SNI HQC, while the NI version of HQC reduces this number to 1975 million cycles.

It is worth noting that the observed difference in cycle counts for encapsulation between HQC and BIKE is due to the intrinsic structural differences between the two schemes. This performance gap is already present in the unmasked reference implementations: according to benchmark data from the official documentations (Table 8 of [AMAB<sup>+</sup>23] and Tables 8 and 9 of [ABB<sup>+</sup>22]), HQC encapsulation is inherently slower by a factor of 2 to 4 compared to BIKE encapsulation. This trend is consistently reflected in the masked implementations.

A significant part of the computation time in HQC comes from the encoding and decoding operations of the public concatenated Reed-Muller and Reed-Solomon codes. It is known that this concatenated construction can be replaced, without compromising security, by an alternative code exhibiting similar properties, particularly the proven low Decryption Failure Rate (DFR). A future work could be to identify a code that remains compatible with the HQC specifications while offering a friendlier structure to masking techniques.

**Scaling** Figure 9 represents the scaling of the number of cycles of our implementation with the masking order. We use the execution time of our order-0 masked implementation as the reference value to construct the plot.

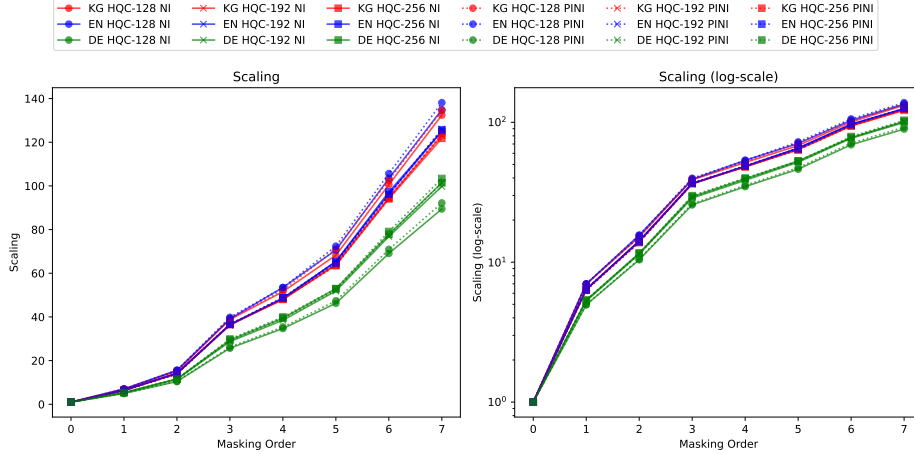


Figure 9: Scaling of the number of cycles with masking order, for all security levels, based on benchmarks with `-O3` optimization flag.

## 6 Conclusion

In this work, we analyzed the side-channel security of the post-quantum standard HQC through a comprehensive sensitivity analysis, to identify and protect future HQC implementations. We show the need for fine-grained protections at all stages of the HQC decoding process and demonstrates that strong provable security against side-channel attacks must

be achieved in practice to secure HQC. This work is grounded in prior research on masked implementations of post-quantum cryptographic schemes, particularly BIKE [DR24] and Dilithium [ABC<sup>+</sup>22b], which served as valuable references for the design and evaluation of our work.

We also provide portable, fully masked, implementations of HQC (formally and practically verified) for its three security levels, in two distinct variants: (i) one adhering to the NI security model, aligned with the state of the art in PQC masking, and (ii) one complying with the MIMO-SNI security model, which offers stronger composability guarantees. To support both variants, we first designed a generic set of dedicated gadgets, which we then secured under each respective security model (NI and MIMO-SNI). The NI variant and its corresponding benchmarks enable fair comparisons with other masked implementations built under the same assumptions. Our stronger MIMO-SNI variant, constructed as a composition of MIMO-SNI-secured gadgets, is, to the best of our knowledge, the first of its kind. We demonstrated that the enhanced security guarantees of this variant incur only a modest performance overhead: for HQC, the performance gap between the two security models is approximately 5% to 10%. Nevertheless, across all evaluated masking orders (0 to 7) and both security models, the execution time of our fully masked HQC remains lower than that of BIKE, the other fully masked code-based implementation.

**Future works** In future work, we could focus on optimizing various aspects of our implementation. Preliminary observations suggest that specific computational routines and function calls may be restructured or refined to reduce execution time and improve overall efficiency. These optimizations could involve algorithmic improvements, or the use of more efficient data structures, all of which have the potential to significantly enhance performance. The focus could also shift towards developing a version with a reduced memory footprint, targeting constrained environments.

As demonstrated in this paper, the overhead introduced when upgrading from NI to MIMO-SNI security remains relatively low—approximately 10%. This observation opens a promising avenue for future work: strengthening the security of existing masked implementations by securing each gadget individually under the MIMO-SNI model and composing them following the required composability rules. Such an approach could significantly improve security guarantees while maintaining acceptable performance levels.

Future work could also explore additional countermeasures, such as shuffling, or propose a fault-resistant implementation.

Another promising research direction would be the development of automated tools to assist in the secure composition of masked algorithms. Such tools could automatically identify appropriate locations for inserting refresh operations and verify that the required composability conditions are satisfied throughout the design. This would enable a more systematic and reliable process for upgrading existing implementations to the MIMO-SNI security model. By ensuring refreshes are placed only where needed, and verifying that all security constraints are correctly enforced, this approach could significantly improve both the efficiency and the robustness of masked implementations.

## 7 Acknowledgments

This work was partially funded by the grant 2022154 from the Appel à projets 2022 thèses AID Cifre-Défense by the Agence de l’Innovation de Défense (AID), Ministère des Armées (French Ministry of Defense). This work was partially funded by the PIA4 X7-PQC project.

## References

- [AAA<sup>+</sup>22] Gorjan Alagic, Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the third round of the nist post-quantum cryptography standardization process. 2022.
- [AAC<sup>+</sup>23] Gorjan Alagic, D Apon, D Cooper, Q Dang, T Dang, J Kelsey, J Lichtinger, C Miller, D Moody, R Peralta, et al. Status report on the fourth round of the nist post-quantum cryptography standardization process, 2023.
- [AAC<sup>+</sup>25] Gorjan Alagic, D Apon, D Cooper, Q Dang, T Dang, J Kelsey, J Lichtinger, C Miller, D Moody, R Peralta, et al. Status report on the fourth round of the nist post-quantum cryptography standardization process, 2025.
- [ABB<sup>+</sup>17] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Shay Gueron, Tim Guneyasu, Carlos Aguilar Melchor, et al. BIKE: Bit Flipping Key Encapsulation. 2017.
- [ABB<sup>+</sup>22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Shay Gueron, Tim Guneyasu, Carlos Aguilar Melchor, et al. BIKE reference implementation, October, 2022. <https://bikesuite.org/#implementation>.
- [ABC<sup>+</sup>22a] Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Markus Schönauer, Tobias Schneider, François-Xavier Standaert, and Christine van Vredendaal. Protecting dilithium against leakage: Revisited sensitivity analysis and improved implementations. *Cryptography ePrint Archive*, 2022.
- [ABC<sup>+</sup>22b] Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Markus Schönauer, Tobias Schneider, François-Xavier Standaert, and Christine van Vredendaal. Protecting dilithium against leakage: Revisited sensitivity analysis and improved implementations. *Cryptography ePrint Archive*, 2022.
- [AGZ20] Nicolas Aragon, Philippe Gaborit, and Gilles Zémor. HQC-RMRS, an instantiation of the HQC encryption framework with a more efficient auxiliary error-correcting code. *arXiv preprint arXiv:2005.10741*, 2020.
- [AMAB<sup>+</sup>17] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, and Gilles Zémor. Hamming Quasi-Cyclic (HQC). 2017.
- [AMAB<sup>+</sup>23] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, and Gilles Zémor. HQC reference implementation, April, 2023. <https://pqc-hqc.org/implementation.html>.
- [BBD<sup>+</sup>16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 116–129, 2016.

- [BC22] Olivier Bronchain and Gaëtan Cassiers. Bitslicing arithmetic/boolean masking conversions for fun and profit: with application to lattice-based kems. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 553–588, 2022.
- [BCL<sup>+</sup>] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic McEliece: conservative code-based cryptography.
- [BCPZ16] Alberto Battistello, Jean-Sébastien Coron, Emmanuel Prouff, and Rina Zeitoun. Horizontal side-channel attacks and countermeasures on the isw masking scheme. In *International Conference on Cryptographic Hardware and Embedded Systems*, pages 23–39. Springer, 2016.
- [BDK<sup>+</sup>18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [BGR<sup>+</sup>21] Joppe W Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. Masking kyber: first-and higher-order implementations, 2021.
- [BHK<sup>+</sup>19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
- [BMG<sup>+</sup>24] Chloé Baisse, Antoine Moran, Guillaume Goy, Julien Maillard, Nicolas Aragon, Philippe Gaborit, Maxime Lecomte, and Antoine Loiseau. Secret and shared keys recovery on hamming quasi-cyclic with sasca. *Cryptology ePrint Archive*, 2024.
- [BMVT78] Elwyn Berlekamp, Robert McEliece, and Henk Van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information Theory*, 24(3):384–386, 1978.
- [CGL<sup>+</sup>24] Jean-Sébastien Coron, François Gérard, Tancrede Lepoint, Matthias Trannoy, and Rina Zeitoun. Improved high-order masked generation of masking vector and rejection sampling in dilithium. *Cryptology ePrint Archive*, 2024.
- [CGTV15] Jean-Sébastien Coron, Johann Großschädl, Mehdi Tibouchi, and Praveen Kumar Vadnala. Conversion from arithmetic to boolean masking with logarithmic complexity. In *Fast Software Encryption: 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers 22*, pages 130–149. Springer, 2015.
- [CGV14] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. Secure conversion between boolean and arithmetic masking of any order. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 188–205. Springer, 2014.
- [CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Transactions on Information Forensics and Security*, 15:2542–2555, 2020.

- [DCE16] A. Adam Ding, Cong Chen, and Thomas Eisenbarth. Simpler, faster, and more robust t-test based leakage detection. In François-Xavier Standaert and Elisabeth Oswald, editors, *Constructive Side-Channel Analysis and Secure Design*, pages 163–183, Cham, 2016. Springer International Publishing.
- [DKL<sup>+</sup>18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.
- [dPKPR24] Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In *Annual International Cryptology Conference*, pages 409–444. Springer, 2024.
- [DR24] Loïc Demange and Mélissa Rossi. A provably masked implementation of bike key encapsulation mechanism. *Cryptology ePrint Archive*, 2024.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.
- [FVBR<sup>+</sup>22] Tim Fritzmann, Michiel Van Beirendonck, Debapriya Basu Roy, Patrick Karl, Thomas Schamberger, Ingrid Verbauwhede, and Georg Sigl. Masked accelerators and instruction set extensions for post-quantum cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 414–460, 2022.
- [Ger24] François Gerard. Dilithim masked implementaion, 2024. [https://github.com/fragerar/tches24\\_masked\\_Dilithium](https://github.com/fragerar/tches24_masked_Dilithium).
- [GJ20] Qian Guo and Thomas Johansson. A new decryption failure attack against hqc. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 353–382. Springer, 2020.
- [GLG22a] Guillaume Goy, Antoine Loiseau, and Philippe Gaborit. A new key recovery side-channel attack on HQC with chosen ciphertext. In *International Conference on Post-Quantum Cryptography*, pages 353–371. Springer, 2022.
- [GLG22b] Guillaume Goy, Antoine Loiseau, and Phippe Gaborit. Estimating the strength of horizontal correlation attacks in the hamming weight leakage model: A side-channel analysis on HQC KEM. In *WCC 2022: The Twelfth International Workshop on Coding and Cryptography*, page WCC\_2022\_paper\_48, 2022.
- [GMGL23] Guillaume Goy, Julien Maillard, Philippe Gaborit, and Antoine Loiseau. Single trace HQC shared key recovery with SASCA. *Cryptology ePrint Archive*, 2023. <https://ia.cr/2023/1590>.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In *Theory of Cryptography Conference*, pages 341–371. Springer, 2017.
- [HKL<sup>+</sup>22] Daniel Heinz, Matthias J Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Amber Sprenkels. First-order masked kyber on arm cortex-m4. *Cryptology ePrint Archive*, 2022.

- [HSC<sup>+</sup>23] Senyang Huang, Rui Qi Sim, Chitchanok Chuengsatiansup, Qian Guo, and Thomas Johansson. Cache-timing attack against HQC. *Cryptology ePrint Archive*, 2023.
- [MGTF19] Vincent Migliore, Benoît Gérard, Mehdi Tibouchi, and Pierre-Alain Fouque. Masking dilithium: Efficient implementation and side-channel evaluation. In *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings 17*, pages 344–362. Springer, 2019.
- [MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error correcting codes*, volume 16. Elsevier, 1977.
- [PFH<sup>+</sup>20] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon. *Post-Quantum Cryptography Project of NIST*, 2020.
- [PRJB] Thales Paiva, Prasanna Ravi, Dirmanto Jap, and Shivam Bhasin. Et tu, brute? side-channel assisted chosen ciphertext attacks using valid ciphertexts.
- [PT19] Thales Bandiera Paiva and Routo Terada. A timing attack on the hqc encryption scheme. In *International Conference on Selected Areas in Cryptography*, pages 551–573. Springer, 2019.
- [SHR<sup>+</sup>22] Thomas Schamberger, Lukas Holzbaur, Julian Renner, Antonia Wachter-Zeh, and Georg Sigl. A power side-channel attack on the reed-muller reed-solomon version of the HQC cryptosystem. In *International Conference on Post-Quantum Cryptography*, pages 327–352. Springer, 2022.
- [SRSWZ20] Thomas Schamberger, Julian Renner, Georg Sigl, and Antonia Wachter-Zeh. A power side-channel attack on the CCA2-secure HQC KEM. In *19th Smart Card Research and Advanced Application Conference (CARDIS2020)*, 2020.
- [SVP<sup>+</sup>25] Maxime Spyropoulos, David Vigilant, Fabrice Perion, Renaud Pacalet, and Laurent Sauvage. Masked vector sampling for HQC. In *Proceedings of the 22nd International Conference on Security and Cryptography - Volume 1: SECRYPT*, pages 750–758. INSTICC, SciTePress, 2025.
- [WTBB<sup>+</sup>20] Guillaume Wafo-Tapa, Slim Bettaleb, Loïc Bidoux, Philippe Gaborit, and Etienne Marcatel. A practicable timing attack against HQC and its counter-measure. *Advances in Mathematics of Communications*, 2020.

## A Additionnal Benchmark with `-O0` optimization flag.

We conducted additionnal benchmarks on a laptop equipped with an Intel Core Ultra 7 165U processor (14 logical CPUs), 16 GB of RAM, a 512 GB SSD, and running a Windows operating system. The processor was configured to maintain a fixed frequency of 2.69GHz, ensuring stable experimental conditions. To measure executions times, we directly accessed the processor’s time-stamp counter using the `rdtsc` instruction, which directly provides the number of CPU clock cycles. We compiled our HQC masked implementations using `gcc 11.4.0` with the `-O0` optimization flag.

For each masking order from 0 to 7 (1 to 8 shares) we repeated the experiment 100 times, preceded by a warm-up step, to obtain an average time for KeyGen, Encaps, and Decaps. We did a benchmark for two different HQC implementations with two security

models, NI and MIMO-SNI, to estimate the impact of increased security on performance. The results of the benchmarks are shown in Table 6 for the MIMO-SNI security model and in Table 7 for the NI security model. We also provide scaled values, with reference 1 for masking order 0, for easier comparison of the impact of masking orders.

The order-0 masking configuration corresponds to an implementation using a single share per variable. While this configuration provides no protection for sensitive variables, it serves as a baseline for comparing execution time scaling across implementations with masking structures.

Table 6: Benchmarks of MIMO-SNI HQC masked implementation (in millions of cycles) with -O0 optimization flag.

| Order         | 0            | 1            | 2            | 3             | 4             | 5             | 6             | 7             |
|---------------|--------------|--------------|--------------|---------------|---------------|---------------|---------------|---------------|
| HQC-128       |              |              |              |               |               |               |               |               |
| KeyGen        | 43.8         | 140.9        | 305.8        | 603.8         | 909.6         | 1224.2        | 1717.5        | 2265.7        |
| KeyGen scaled | $\times 1.0$ | $\times 3.2$ | $\times 7.0$ | $\times 13.8$ | $\times 20.8$ | $\times 28.0$ | $\times 39.2$ | $\times 51.8$ |
| Encaps        | 79.3         | 263.7        | 571.1        | 1120.6        | 1696.6        | 2289.0        | 3217.5        | 4232.4        |
| Encaps scaled | $\times 1.0$ | $\times 3.3$ | $\times 7.2$ | $\times 14.1$ | $\times 21.4$ | $\times 28.8$ | $\times 40.6$ | $\times 53.3$ |
| Decaps        | 132.3        | 414.6        | 885.4        | 1732.9        | 2605.5        | 3497.4        | 4917.5        | 6455.8        |
| Decaps scaled | $\times 1.0$ | $\times 3.1$ | $\times 6.7$ | $\times 13.1$ | $\times 19.7$ | $\times 26.4$ | $\times 37.2$ | $\times 48.8$ |
| HQC-192       |              |              |              |               |               |               |               |               |
| KeyGen        | 121.9        | 389.9        | 862.8        | 1691.7        | 2497.8        | 3466.0        | 4871.1        | 6371.4        |
| KeyGen scaled | $\times 1.0$ | $\times 3.2$ | $\times 7.1$ | $\times 13.9$ | $\times 20.5$ | $\times 28.4$ | $\times 40.0$ | $\times 52.3$ |
| Encaps        | 221.8        | 733.3        | 1610.8       | 3126.5        | 4680.7        | 6450.7        | 9074.0        | 11893.3       |
| Encaps scaled | $\times 1.0$ | $\times 3.3$ | $\times 7.3$ | $\times 14.1$ | $\times 21.1$ | $\times 29.1$ | $\times 40.9$ | $\times 53.6$ |
| Decaps        | 338.2        | 1077.5       | 2352.1       | 4593.9        | 6867.2        | 9480.0        | 13263.3       | 17245.3       |
| Decaps scaled | $\times 1.0$ | $\times 3.2$ | $\times 7.0$ | $\times 13.6$ | $\times 20.3$ | $\times 28.0$ | $\times 39.2$ | $\times 51.0$ |
| HQC-256       |              |              |              |               |               |               |               |               |
| KeyGen        | 233.0        | 758.7        | 1655.9       | 3292.2        | 4880.5        | 6693.6        | 9198.3        | 12179.9       |
| KeyGen scaled | $\times 1.0$ | $\times 3.3$ | $\times 7.1$ | $\times 14.1$ | $\times 20.9$ | $\times 28.7$ | $\times 39.5$ | $\times 52.3$ |
| Encaps        | 429.6        | 1401.1       | 3035.6       | 6073.3        | 8958.1        | 12291.4       | 17056.4       | 22614.3       |
| Encaps scaled | $\times 1.0$ | $\times 3.3$ | $\times 7.1$ | $\times 14.1$ | $\times 20.9$ | $\times 28.6$ | $\times 39.7$ | $\times 52.6$ |
| Decaps        | 646.0        | 2056.2       | 4433.9       | 8794.2        | 12998.6       | 17867.4       | 24738.1       | 32703.0       |
| Decaps scaled | $\times 1.0$ | $\times 3.2$ | $\times 6.9$ | $\times 13.6$ | $\times 20.1$ | $\times 27.7$ | $\times 38.3$ | $\times 50.6$ |

**Scaling** Figure 10 represents the scaling of the number of cycles of our implementation with the masking order. We use the execution time of our order-0 masked implementation as the reference value to construct the plot.



Table 7: Benchmarks of NI HQC masked implementation (in millions of cycles) with -O0 optimization flag.

| Order         | 0            | 1            | 2            | 3             | 4             | 5             | 6             | 7             |
|---------------|--------------|--------------|--------------|---------------|---------------|---------------|---------------|---------------|
| HQC-128       |              |              |              |               |               |               |               |               |
| KeyGen        | 43.5         | 137.0        | 296.9        | 584.5         | 867.5         | 1195.1        | 1664.4        | 2254.5        |
| KeyGen scaled | $\times 1.0$ | $\times 3.1$ | $\times 6.8$ | $\times 13.4$ | $\times 19.9$ | $\times 27.5$ | $\times 38.3$ | $\times 51.8$ |
| Encaps        | 78.5         | 252.3        | 549.5        | 1079.9        | 1607.8        | 2230.7        | 3118.9        | 4158.8        |
| Encaps scaled | $\times 1.0$ | $\times 3.2$ | $\times 7.0$ | $\times 13.8$ | $\times 20.5$ | $\times 28.4$ | $\times 39.7$ | $\times 53.0$ |
| Decaps        | 132.1        | 402.8        | 854.3        | 1665.5        | 2501.5        | 3438.6        | 4770.1        | 6317.3        |
| Decaps scaled | $\times 1.0$ | $\times 3.1$ | $\times 6.5$ | $\times 12.6$ | $\times 18.9$ | $\times 26.0$ | $\times 36.1$ | $\times 47.8$ |
| HQC-192       |              |              |              |               |               |               |               |               |
| KeyGen        | 119.6        | 387.6        | 832.4        | 1660.5        | 2431.6        | 3370.0        | 4883.0        | 6194.0        |
| KeyGen scaled | $\times 1.0$ | $\times 3.2$ | $\times 7.0$ | $\times 13.9$ | $\times 20.3$ | $\times 28.2$ | $\times 40.8$ | $\times 51.8$ |
| Encaps        | 217.4        | 713.5        | 1559.3       | 3068.7        | 4480.5        | 6321.9        | 8880.4        | 11539.9       |
| Encaps scaled | $\times 1.0$ | $\times 3.3$ | $\times 7.2$ | $\times 14.1$ | $\times 20.6$ | $\times 29.1$ | $\times 40.9$ | $\times 53.1$ |
| Decaps        | 329.8        | 1054.5       | 2277.5       | 4458.1        | 6531.2        | 9200.9        | 12950.8       | 16753.3       |
| Decaps scaled | $\times 1.0$ | $\times 3.2$ | $\times 6.9$ | $\times 13.5$ | $\times 19.8$ | $\times 27.9$ | $\times 39.3$ | $\times 50.8$ |
| HQC-256       |              |              |              |               |               |               |               |               |
| KeyGen        | 232.6        | 731.7        | 1625.9       | 3190.0        | 4759.8        | 6434.5        | 9058.0        | 11870.6       |
| KeyGen scaled | $\times 1.0$ | $\times 3.1$ | $\times 7.0$ | $\times 13.7$ | $\times 20.5$ | $\times 27.7$ | $\times 38.9$ | $\times 51.0$ |
| Encaps        | 419.7        | 1346.7       | 2981.0       | 5837.9        | 8735.0        | 11988.8       | 16809.1       | 21892.5       |
| Encaps scaled | $\times 1.0$ | $\times 3.2$ | $\times 7.1$ | $\times 13.9$ | $\times 20.8$ | $\times 28.6$ | $\times 40.1$ | $\times 52.2$ |
| Decaps        | 630.7        | 1995.6       | 4349.7       | 8494.0        | 12644.3       | 17327.5       | 24222.1       | 31732.0       |
| Decaps scaled | $\times 1.0$ | $\times 3.2$ | $\times 6.9$ | $\times 13.5$ | $\times 20.0$ | $\times 27.5$ | $\times 38.4$ | $\times 50.3$ |

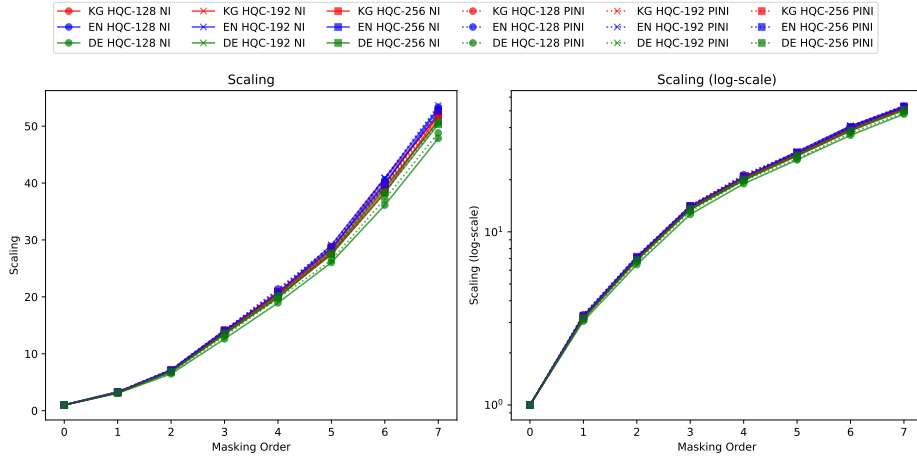


Figure 10: Scaling of the number of cycles with masking order, for all security levels, based on benchmark with -O0 optimization flag.

## B A new theoretical side-channel approach to target the Reed-Solomon decoder

In this section, we introduce a new side-channel attack that exploits the leakage of Reed-Solomon decoding to recover the secret key of HQC. Existing attacks rely on the inner Reed-Muller code to extract information. Our new attack is the first to exploit the outer Reed-Solomon code for key recovery. It is based on the fact that the Reed-Solomon decoder corrects the errors that the Reed-Muller code failed to correct. As a result, analyzing the behavior of the Reed-Solomon decoder can reveal whether the Reed-Muller decoding was successful or not. Based on this insight, we construct a novel two-steps attack: (i) using the leakage observed during Reed-Solomon decoding, we design a decoding failure oracle for the Reed-Muller code; (ii) we then leverage this oracle to mount a Chosen Ciphertext Attack (CCA) that allows the recovery of the  $\mathbf{y}$  part of the secret key.

### B.1 Hamming weight oracle

When the Reed-Muller decoder fails to correct all errors, the Reed-Solomon decoder operates on non-zero data. This allows us to construct an oracle  $\mathcal{O}(\cdot)$  that determines whether the Reed-Muller decoder successfully corrected all errors or not.

**Practical validation of the oracle** We acquired traces of power consumption of a STM32F407 micro-controller during the Reed-Solomon decoding, without residual errors (trace set  $\mathcal{S}_0$ ) and with residual errors (trace set  $\mathcal{S}_1$ ).  $\mathcal{S}_0$  is the set of traces for which the Reed-Muller code corrected all errors, the Reed-Solomon code operates with zero data.  $\mathcal{S}_1$  is the set of traces for which the Reed-Muller code left some remaining errors, the Reed-Solomon code operates with nonzero data. We then applied a Test Vector Leakage Assessment (TVLA) using the Welch’s  $t$ -test which results are shown in Figure 11.

To perform this analysis, we use the Welch’s  $t$ -test described in Equation 4. Results are plotted in Figure 11 and show that the threshold is exceeded in many samples. It follows that the two sets are statistically distinguishable with a high level of confidence, which suggests that a classifier could be constructed based on these traces to detect Reed-Muller decoding failures.

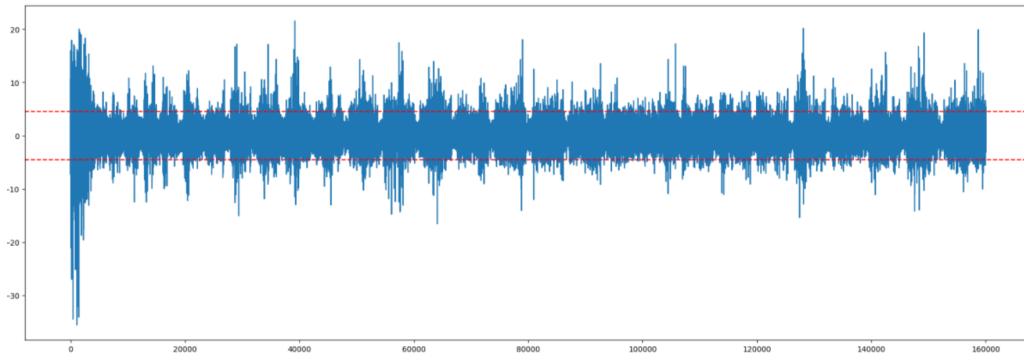


Figure 11:  $t$ -test on two sets of 100 traces each of the Reed-Solomon decoding, 4.5 threshold plotted with dashed red lines.

### B.2 Recovering the message from the oracle

We suppose that our oracle  $\mathcal{O}(\cdot)$  of Reed-Muller decoding failures is perfect. We describe now how to exploit it to recover the  $\mathbf{y}$  part of the HQC secret key. We first recall that the

Reed-Muller code of HQC is indeed made of  $n_1$  independent Reed-Muller codes. Our new attack can be performed independently on each of the  $n_1$  Reed-Muller decoders. Indeed, a decoding failure in a single of these decoders leads to an erroneous codeword at the input of the Reed-Solomon code.

As is done in the state of the art, we consider a ciphertext of the form  $(\mathbf{u}, \mathbf{v}) = (1, \mathbf{v})$ , such that the HQC decoder decodes  $\mathbf{v} - \mathbf{y}$ . Because  $\mathbf{y}$  has a small Hamming weight, the Reed-Muller decoder decodes it as the zero codeword, which is also a Reed-Solomon error-free codeword. We consider this as a **Decoding Success** case.

As the Reed-Muller code is public, and its parameters known, it is easy to chose vector  $\mathbf{v}$  just above the Reed-Muller error Correcting Capability (CC):  $\mathcal{O}(\mathbf{v}) = \text{Decoding Failure}$ . We then call our oracle to obtain  $\mathcal{O}(\mathbf{v} - \mathbf{y})$  with two possible outcomes:

- $\mathcal{O}(\mathbf{v} - \mathbf{y}) = \text{Decoding Failure}$  means that the Hamming weight of  $\mathbf{v} - \mathbf{y}$  is larger than the Reed-Muller error CC.
- $\mathcal{O}(\mathbf{v} - \mathbf{y}) = \text{Decoding Success}$  means that the Hamming weight of  $\mathbf{v} - \mathbf{y}$  is smaller than the Reed-Muller error CC.

In the second case, since decoding  $\mathbf{v}$  alone leads into a **Decoding Failure**, we know that  $\mathbf{v} - \mathbf{y}$  has a smaller Hamming weight than  $\mathbf{v}$ , in other words  $\mathbf{v} \cap \mathbf{y} \neq \emptyset$ .

**Find collisions with  $\mathbf{y}$**  Table 8 shows the parameters of the Reed-Muller code for the different HQC security levels. The codes are specified by their length  $n$ , dimension  $k$  and minimal distance  $d$  using the common  $[n, k, d]$  notation.

Table 8: Parameters of HQC Reed-Muller codes

| Security | Non-duplicated code | Multiplicity | Duplicated code |
|----------|---------------------|--------------|-----------------|
| HQC-128  | [128, 8, 64]        | 3            | [384, 8, 192]   |
| HQC-192  | [128, 8, 64]        | 5            | [640, 8, 320]   |
| HQC-256  | [128, 8, 64]        | 5            | [640, 8, 320]   |

The input  $\mathbf{v} - \mathbf{y}$  of the Reed-Muller decoder is a vector of length  $\text{mul} \times 128$ , where  $\text{mul} = 3$  or  $5$  depending on the security level (see Table 8). It is composed of  $\text{mul}$  blocks of length 128, and is first transformed by **ExpandAndSum** into a 128-components vector  $\mathbf{z} = \text{ExpandAndSum}(\mathbf{v} - \mathbf{y})$  where the 3 or 5 multiplicity has been removed. The transformation is defined component-wise as:

$$z_j = \sum_{i=1}^{\text{mul}} (\mathbf{v} - \mathbf{y})_{i \cdot 128 + j} \quad (5)$$

Since the multiplicity is removed with a maximum-likelihood decoding strategy, for which no exact formula exists, we cannot precisely determine the decoding pattern of the duplicated Reed-Muller code. However, we can do this for the non-duplicated Reed-Muller code used after **ExpandAndSum**:  $\mathbf{z}$  is further processed by the Reed-Muller decoder that detects an error if any component of  $\mathbf{z}$  is different from 0 and  $\text{mul}$ .

The minimal distance  $d$  of the Reed-Muller codes does not depend on the security level, it is always 64. So their error CC is always  $\lfloor \frac{d}{2} \rfloor - 1 = 31$ . We can leverage this to construct our chosen-ciphertext attack and identify the non-zero coordinates of  $\mathbf{y}$  with a dichotomous strategy.

To build vector  $\mathbf{v}$  we first select a set  $\mathcal{I}$  of 32 positions from 128. Next we choose a set  $\mathcal{B}$  of  $\lfloor \frac{\text{mul}}{2} \rfloor + 1$  blocks of  $\mathbf{v}$  and in each of these blocks, we set to 1 the locations in  $\mathcal{I}$ .

$$v_i = \begin{cases} 1 & \text{if } i \bmod 128 \in \mathcal{I} \text{ and } \left\lfloor \frac{i}{\text{mul}} \right\rfloor \in \mathcal{B} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

In this way, the result of  $\text{ExpandAndSum}(\mathbf{v})$  contains exactly 32 entries equal to  $\lfloor \frac{\text{mul}}{2} \rfloor + 1$ , while all other entries are zero. This corresponds to an error pattern that is just above the Reed-Muller error CC, which is limited to correcting up to  $\lfloor \frac{d}{2} \rfloor - 1 = 31$  errors.

**Rebuild  $\mathbf{y}$  from collisions** We can now consider the different possible interactions between  $\mathbf{y}$  and the support of  $\mathbf{v}$ . Let us denote  $l$  the location of a non-zero coordinate of  $\mathbf{y}$ :

- If  $l \in \text{Supp}(\mathbf{v})$ , the corresponding coordinates of  $\mathbf{z} = \text{ExpandAndSum}(\mathbf{v} - \mathbf{y})$  is decreased by 1, bringing the error vector below the Reed-Muller error CC. Hence,  $\mathcal{O}(\mathbf{v} - \mathbf{y}) = \text{Decoding Success}$
- If  $l \bmod 128 \in \mathcal{I}$  but  $l \notin \text{Supp}(\mathbf{v})$ , the error pattern remains uncorrectable: the Hamming weight of  $\mathbf{v}$  remains unchanged, and its non-zero components are farther from zero, leading to a decoding failure:  $\mathcal{O}(\mathbf{v} - \mathbf{y}) = \text{Decoding Failure}$ .
- If  $l \bmod 128 \notin \mathcal{I}$ , vector  $\mathbf{z}$  has an even larger Hamming weight, staying above the Reed-Muller error CC, and leading again to a decoding failure:  $\mathcal{O}(\mathbf{v} - \mathbf{y}) = \text{Decoding Failure}$

We represent the three possible cases on Figure 12. By calling the oracle for each set  $\mathcal{B}$  for a given set  $\mathcal{I}$ , we obtain a set of equations describing the support of  $\mathbf{y}$ . It follows that, by collecting enough of such equations, we are able to rebuild the  $\mathbf{y}$  part of the secret key.

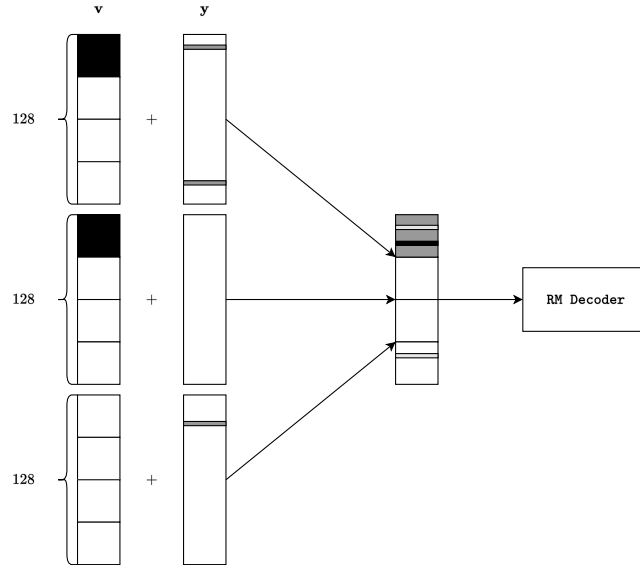


Figure 12: Dichotomous choice of ciphertext  $\mathbf{v}$  to create controlled decoding errors

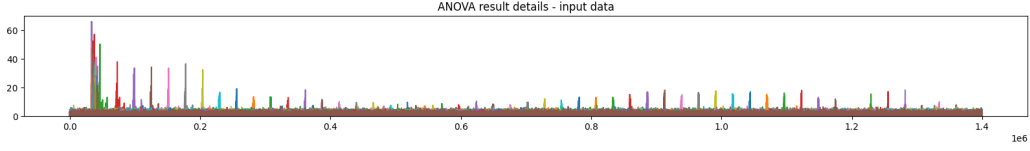
**Attack complexity** We can estimate the number of oracle calls, and therefore the number of physical measurements that are required to perform our attack. Let us denote  $k$  the number of traces required for a perfect accuracy of our oracle (this value depends on the oracle construction, noise level and physical target). We attack the  $n_1$  independent Reed-Muller decoders. A single step of the aforementioned attack provides one equation about a  $n_2$ -bits secret, so  $n_2$  independent equations are required to recover  $\mathbf{y}$ . Each step requires to test  $\binom{\text{mul}}{\lfloor \frac{\text{mul}}{2} \rfloor + 1}$  different sets  $\mathcal{B}$  which ends up with  $T$  required traces:

$$T = k \times n_1 \times n_2 \times \left( \left\lfloor \frac{\text{mul}}{2} \right\rfloor + 1 \right) \quad (7)$$

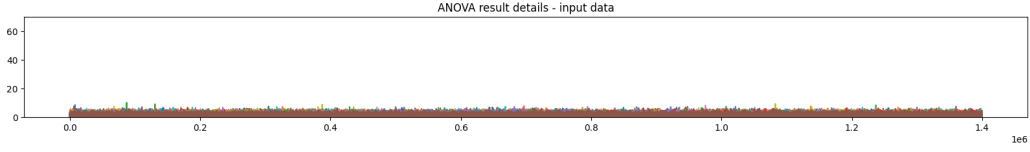
For HQC-128, this represents a total of  $k \times 2^{15.7}$  traces. This number of required traces is larger than with other attacks from the state of the art [BMG<sup>+</sup>24] which can perform a key recovery with a single trace. But we are also competitive with other attacks, including Guo et al. [GJ20] which require more than  $2^{64}$  physical measurements. Nevertheless, the purpose of this discussion is not to build the most efficient attack, but to show that the Reed-Solomon decoding step must be entirely protected to prevent side-channel leakages.

## C ANOVA analysis of the masked Reed-Solomon decoder

This ANOVA leakage analysis complements the  $t$ -test we performed on the RS decoder. At the input of the decoder, the codeword can be viewed as 40 blocks of 8 bits (i.e., 40 bytes). For this analysis, we targeted the Hamming weight of each of these 40 bytes individually. Once again, it demonstrates that the leakage is mitigated by the use of random masks, reinforcing the relevance of our countermeasure.



(a) with masks set to zero.



(b) with random masks.

Figure 13: ANOVA of the masked RS decoder